

คู่มือการสร้างและพัฒนา ESP32-S3 Tailscale VPN + MQTT Sensor Gateway

อุปกรณ์สะพานเครือข่าย VPN พร้อมฟีเจอร์ Auto-Recovery และหน้าจอ OLED

โปรเจกต์: esp_tailscale
แพลตฟอร์ม: ESP32-S3 (ESP-IDF + Arduino Framework)
เวอร์ชันโค้ด: 1.1 (เพิ่มระบบเชื่อมต่อใหม่อัตโนมัติ)
วันที่จัดทำเอกสาร: 24 มิถุนายน 2569

1. บทนำและฟังก์ชันการทำงาน

อุปกรณ์ ESP32-S3 Tailscale VPN + MQTT Sensor Gateway ทำหน้าที่เป็นอุปกรณ์ตัวกลางหลัก (Gateway) ที่ช่วยเชื่อมต่อระบบควบคุมอัตโนมัติภายในโรงงาน/บ้านเรือน เข้ากับเครือข่ายควบคุมส่วนตัวระยะไกลผ่านระบบ VPN ของ Tailscale โดยตัวอุปกรณ์สามารถทำงานโดยตรงจากอินเทอร์เน็ต WiFi ในบ้าน และเชื่อมต่อเข้ากับกลุ่ม Tailnet ส่วนตัวได้ทันทีโดยไม่ต้องตั้งค่าเราเตอร์ (No Port Forwarding)

ฟีเจอร์หลักของอุปกรณ์:

- Tailscale VPN Client: รันไคลเอนต์ VPN ในตัวโดยใช้โมดูล MicroLink รองรับการเข้ารหัสข้อมูล WireGuard แบบฮาร์ดแวร์
- Local MQTT Broker: รัน PicoMQTT Server ภายในบอร์ด เพื่อรับส่งข้อมูลกับโหนดลูก (Client Nodes) ในวง LAN ท้องถิ่น
- Cloud MQTT Bridge: ทำการ Forward สถานะเซนเซอร์และคำสั่งควบคุมไปยัง Cloud MQTT Broker ภายนอกเพื่อเชื่อมกับระบบควบคุมส่วนกลาง
- OLED Display: แสดงสถานะการเชื่อมต่อ IP Address, VPN IP และจำนวนโหนดลูกที่ทำงานอยู่แบบเรียลไทม์
- WS2812B RGB Status LED: แสดงสถานะการทำงานผ่านสีของไฟ LED ช่วยวิเคราะห์ปัญหาฮาร์ดแวร์เบื้องต้น
- Auto-Recovery System: เพิ่มเต็มระบบตรวจสอบและกู้คืนการเชื่อมต่อเครือข่าย WiFi และช็อกเก็ต Tailscale อัตโนมัติ (Anti-Drop) ทำให้ตัวอุปกรณ์สามารถทำงานได้อย่างเสถียรแบบต่อเนื่อง 24 ชั่วโมงต่อวัน โดยไม่ต้องทำการกด Reset ด้วยปุ่มฮาร์ดแวร์ในกรณีที่เครือข่ายล่มชั่วคราว

2. การเชื่อมต่อฮาร์ดแวร์ (Hardware Pinout)

อุปกรณ์/ฟังก์ชัน	รายละเอียดการเชื่อมต่อ	พิน GPIO บน ESP32-S3
WS2812B RGB LED	ไฟ LED แสดงสถานะการเชื่อมต่อบนบอร์ด	GPIO 48 (neopixelWrite)
BOOT Button	ปุ่มกดยืนยันการรีเซ็ตค่าโรงงาน (Active-Low)	GPIO 0
OLED SSD1306 SDA	สายรับส่งข้อมูลหน้าจอแสดงผล I2C	GPIO 8
OLED SSD1306 SCL	สายสัญญาณนาฬิกาหน้าจอแสดงผล I2C	GPIO 9

ระบบรหัสสีไฟสถานะ LED (RGB Status LED):

- **ไฟกะพริบสีน้ำเงิน (ทุก 500ms):**
บอร์דרันอยู่ในโหมด Access Point ตั้งค่า (AP Portal Mode)
- **ไฟกะพริบสีส้ม/เหลือง (ทุก 500ms):**
บอร์ดกำลังทำการเชื่อมต่อเชื่อมกับเน็ตเวิร์ก WiFi STA
- **ไฟสีเขียวค้าง (Solid Green):**
บอร์ดต่อ WiFi สำเร็จแล้ว แต่ VPN ของ Tailscale ยังไม่ทำงานหรืออยู่ในระหว่างเชื่อมต่อ
- **ไฟสีฟ้าค้าง (Solid Cyan):**
บอร์ดต่อ WiFi สำเร็จ และ VPN Tunnel ของ Tailscale ทำงานออนไลน์สำเร็จแล้ว (พร้อมใช้งาน)
- **ไฟกะพริบสีส้ม/แดงอย่างรวดเร็ว (ทุก 100ms):**
ระบบแจ้งเตือนการกดปุ่ม BOOT ค้างเพื่อทำการรีเซ็ตค่าโรงงาน
- **ไฟกะพริบสีแดงรวดเร็ว (2 วินาที):**
ระบบตอบรับการทำ Factory Reset สำเร็จ และกำลังเคลียร์ข้อมูล NVS เก่า

3. โครงสร้างและตั้งค่าโครงการ (Project Setup)

โครงการนี้สร้างขึ้นบนระบบ PlatformIO IDE โดยใช้ Arduino Framework ร่วมกับ ESP-IDF Components เพื่อให้สามารถรันฟังก์ชันการเชื่อมต่อ VPN แบบระดับต่ำ (Low-level C) ได้อย่างเสถียร มีไฟล์ตั้งค่าหลัก 2 ไฟล์ ดังนี้:

ไฟล์ platformio.ini (การตั้งค่าบอร์ดและไลบรารี):

```
; PlatformIO Project Configuration File
;
; Build options: build flags, source filter
; Upload options: custom upload port, speed and extra flags
; Library options: dependencies, extra library storages
; Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino, espidf
monitor_speed = 115200
board_build.filesystem = littlefs
board_build.partitions = default_8MB.csv ; Partition VPN
Flash
build_flags =
```

```
-DBOARD_HAS_PSRAM
-DCONFIG_SPIRAM_SUPPORT
-DCONFIG_LITTLEFS_PAGE_SIZE=256
-Icomponents/littlefs/include
-Icomponents/littlefs/src/littlefs

lib_deps =
  LittleFS
  FS
  PicoMQTT
  Adafruit GFX Library
  Adafruit SSD1306
```

ไฟล์ sdkconfig.defaults (การตั้งค่าระดับลึกสำหรับ ESP-IDF และ PSRAM):

```
CONFIG_FREERTOS_HZ=1000
CONFIG_ESP32S3_SPIRAM_SUPPORT=y
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_SPIRAM_MODE_QUAD=n
CONFIG_SPIRAM_SPEED_80M=y
CONFIG_SPIRAM_TYPE_AUTO=y
CONFIG_ESPTOOLPY_FLASHSIZE_8MB=y
CONFIG_ESPTOOLPY_FLASHSIZE="8MB"
CONFIG_AUTOSTART_ARDUINO=y
CONFIG_MBEDTLS_CHACHA20_C=y
CONFIG_MBEDTLS_POLY1305_C=y
CONFIG_MBEDTLS_CHACHAPOLY_C=y
CONFIG_LITTLEFS_PAGE_SIZE=256
CONFIG_LITTLEFS_BLOCK_SIZE=4096
CONFIG_LITTLEFS_LOOKAHEAD_SIZE=128
CONFIG_ARDUINO_LOOP_STACK_SIZE=32768
```

4. กลไก Auto-Recovery (การตรวจจับและเชื่อมต่อใหม่อัตโนมัติ)

เพื่อให้บอร์ดสามารถทำงานได้อย่างเสถียรตลอด 24 ชั่วโมง เราได้เพิ่มฟังก์ชัน `checkConnectionRecovery()` เข้าไปในกระบวนการทำงานหลัก (loop) เพื่อแก้ไขปัญหาที่มักเกิดขึ้นตามปกติ:

1. WiFi Connection Monitor: ตรวจจับสถานะการเชื่อมต่อ WiFi ทุกๆ 5 วินาที หากหลุดระบบจะเรียกใช้ `WiFi.begin()` เพื่อเชื่อมต่อใหม่เป็นระยะๆ (ทุกๆ 15 วินาที)
2. WiFi Stack Hard Reset: หากสัญญาณหลุดเกิน 60 วินาที ระบบจะปลดการทำงานทั้งหมดและปิด-เปิดโหมด WiFi ใหม่ เพื่อขจัดสถานะค้าง (Driver Lockup)
3. Tailscale Sockets Rebind: เมื่อเชื่อมต่อ WiFi กลับเข้าสู่ระบบได้สำเร็จ ระบบจะส่งคำสั่ง `microlink_rebind(ml)` ไปยังตัวจำลอง VPN เพื่อให้ปิดข้อก่เกิดการคุย Wireguard/DERP อันเก่าและเริ่มทำงานเชื่อมต่อกับเกตเวย์ตัวกลางของ VPN ใหม่ทันทีโดยไม่ต้องทำการรีบูตทั้งระบบ (รักษาเสถียรภาพของ Local MQTT Broker)

5. โค้ดโปรแกรมฉบับเต็มอย่างละเอียด (Full Source Code - main.c)

นี่คือโค้ดโปรแกรมทั้งหมดของบอร์ด ESP32-S3 Gateway ที่สมบูรณ์พร้อมระบบจัดสรร OLED หน้าสัมผัสปุ่มรีเซ็ต และฟังก์ชันตรวจสอบการเชื่อมต่อ WiFi / VPN:

```

1: #include <Arduino.h>
2: #include <WiFi.h>
3: #include <LittleFS.h>
4: #include <WebServer.h>
5: #include <Wire.h>
6: #include <Adafruit_GFX.h>
7: #include <Adafruit_SSD1306.h>
8: #include <PicoMQTT.h>
9: #include <ESPmDNS.h>
10:
11: // MicroLink C Component (Tailscale Client)
12: extern "C" {
13:     #include "microlink.h"
14: }
15:
16: // =====
17: // 1. Hardware Pin Definitions
18: // =====
19: const int LED_PIN = 48;           // GPIO 48: Built-in RGB LED on ESP32-S3 DevKit (RGB@IO48)
20: const int RESET_BTN_PIN = 0;     // GPIO 0: BOOT button
21: const int OLED_SDA_PIN = 8;      // I2C SDA = 8
22: const int OLED_SCL_PIN = 9;      // I2C SCL = 9
23:
24: // =====
25: // 2. Configuration Parameters
26: // =====
27: String wifi_ssid = "";
28: String wifi_pass = "";
29: String node_id = "esp32-gateway";
30: String tailscale_auth_key = ""; // Tailscale Auth Key (tskey-auth-...)
31: String registered_auth_key = ""; // Last registered Auth Key (NVS checker)
32: String cloud_server = "broker.emqx.io";
33: int cloud_port = 1883;
34: String cloud_user = "allarm";
35: String cloud_pass = "123456";
36:
37: bool isAPMode = false;
38: bool isTailscaleStarted = false;
39:
40: // MicroLink instance pointer
41: microlink_t *ml = nullptr;
42:
43: // =====
44: // 3. OLED Display Config (SSD1306)
45: // =====
46: #define SCREEN_WIDTH 128
47: #define SCREEN_HEIGHT 64
48: #define OLED_RESET -1
49: #define SCREEN_ADDRESS 0x3C
50: Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
51:
52: // =====

```

```
53: // 4. Dynamic Sensor Nodes Structures
54: // =====
55: #define MAX_SENSORS_PER_NODE 8
56: #define MAX_NODES 8
57:
58: struct SensorReading {
59:     String key;
60:     float value = 0.0;
61:     bool active = false;
62: };
63:
64: struct Node {
65:     String id;
```

```

66:   SensorReading sensors[MAX_SENSORS_PER_NODE];
67:   bool out_states[5] = {false, false, false, false, false};
68:   unsigned long last_seen = 0;
69:   bool active = false;
70: };
71:
72: Node nodes[MAX_NODES];
73: int active_node_count = 0;
74:
75: // Search or register a client node
76: int getOrRegisterNode(String nodeId) {
77:   for (int i = 0; i < MAX_NODES; i++) {
78:     if (nodes[i].active && nodes[i].id == nodeId) {
79:       nodes[i].last_seen = millis();
80:       return i;
81:     }
82:   }
83:   for (int i = 0; i < MAX_NODES; i++) {
84:     if (!nodes[i].active) {
85:       nodes[i].id = nodeId;
86:       nodes[i].active = true;
87:       for (int j = 0; j < MAX_SENSORS_PER_NODE; j++) {
88:         nodes[i].sensors[j].active = false;
89:       }
90:       memset(nodes[i].out_states, 0, sizeof(nodes[i].out_states));
91:       nodes[i].last_seen = millis();
92:       active_node_count++;
93:       return i;
94:     }
95:   }
96:   return -1;
97: }
98:
99: // Update sensor readings dynamically
100: void updateNodeSensor(int nodeId, String sensorKey, float val) {
101:   for (int i = 0; i < MAX_SENSORS_PER_NODE; i++) {
102:     if (nodes[nodeId].sensors[i].active && nodes[nodeId].sensors[i].key == sensorKey) {
103:       nodes[nodeId].sensors[i].value = val;
104:       return;
105:     }
106:   }
107:   for (int i = 0; i < MAX_SENSORS_PER_NODE; i++) {
108:     if (!nodes[nodeId].sensors[i].active) {
109:       nodes[nodeId].sensors[i].key = sensorKey;
110:       nodes[nodeId].sensors[i].value = val;
111:       nodes[nodeId].sensors[i].active = true;
112:       Serial.printf("[GATEWAY] Node '%s' registered new sensor key: '%s' = %.2f\n",
113:         nodes[nodeId].id.c_str(), sensorKey.c_str(), val);
114:       return;
115:     }
116:   }
117: }
118:
119: // =====
120: // 5. PicoMQTT Declarations
121: // =====
122: PicoMQTT::Server* local_broker = nullptr;
123: PicoMQTT::Client* cloud_client = nullptr;
124: WebServer server(80);
125:

```



```
126: // =====
127: // 6. RGB LED Debug Helper (neopixelWrite)
128: // =====
129: void setLedColor(uint8_t r, uint8_t g, uint8_t b) {
130:   neopixelWrite(LED_PIN, r, g, b);
```

```

131: }
132:
133: // =====
134: // 7. OLED Display Helper
135: // =====
136: void updateOLED(String statusMsg) {
137:   display.clearDisplay();
138:   display.setTextCursor(SSD1306_WHITE);
139:   display.setTextSize(1);
140:   display.setCursor(0, 0);
141:
142:   if (isAPMode) {
143:     display.println("--- SETUP PORTAL ---");
144:     display.setCursor(0, 16);
145:     display.print("SSID: Gateway-");
146:     display.println(node_id);
147:     display.setCursor(0, 28);
148:     display.print("IP: 192.168.4.1");
149:   } else {
150:     display.println("--- MQTT & VPN GW ---");
151:     display.setCursor(0, 14);
152:     display.print("IP: ");
153:     display.println(WiFi.localIP().toString());
154:
155:     display.setCursor(0, 24);
156:     display.print("VPN: ");
157:     if (isTailscaleStarted && ml != nullptr && microlink_is_connected(ml)) {
158:       uint32_t vpn_ip = microlink_get_vpn_ip(ml);
159:       char ip_str[16];
160:       microlink_ip_to_str(vpn_ip, ip_str);
161:       display.println(ip_str);
162:     } else {
163:       display.println("Connecting...");
164:     }
165:
166:     display.setCursor(0, 36);
167:     display.printf("Nodes Active: %d/%d", active_node_count, MAX_NODES);
168:   }
169:
170:   display.setCursor(0, 46);
171:   if (!isAPMode && active_node_count > 0) {
172:     for (int i = 0; i < MAX_NODES; i++) {
173:       if (nodes[i].active) {
174:         String firstSensorName = "";
175:         float firstSensorVal = 0;
176:         for (int j = 0; j < MAX_SENSORS_PER_NODE; j++) {
177:           if (nodes[i].sensors[j].active) {
178:             firstSensorName = nodes[i].sensors[j].key;
179:             firstSensorVal = nodes[i].sensors[j].value;
180:             break;
181:           }
182:         }
183:         if (firstSensorName != "") {
184:           display.printf("%s:%s=%.1f", nodes[i].id.c_str(), firstSensorName.c_str(),
firstSensorVal);
185:         } else {
186:           display.printf("Node %s online", nodes[i].id.c_str());
187:         }
188:         break;
189:       }

```

```
190:     }
191:   } else if (!isAPMode) {
192:     display.print("Waiting for Nodes...");
193:   } else {
194:     display.print("Connect & configure");
195:   }
```

```
196:
197:   display.setCursor(0, 56);
198:   display.print("Status: ");
199:   display.print(statusMsg);
200:
201:   display.display();
202: }
203:
204: // =====
205: // 8. Configuration Load/Save from LittleFS
206: // =====
207: void loadConfig() {
208:   if (LittleFS.exists("/config.txt")) {
209:     File configFile = LittleFS.open("/config.txt", "r");
210:     if (configFile) {
211:       while (configFile.available()) {
212:         String line = configFile.readStringUntil('\n');
213:         line.trim();
214:         int separatorIdx = line.indexOf('=');
215:         if (separatorIdx != -1) {
216:           String key = line.substring(0, separatorIdx);
217:           String val = line.substring(separatorIdx + 1);
218:
219:           if (key == "wifi_ssid") wifi_ssid = val;
220:           else if (key == "wifi_pass") wifi_pass = val;
221:           else if (key == "node_id") node_id = val;
222:           else if (key == "tailscale_auth_key") tailscale_auth_key = val;
223:           else if (key == "registered_auth_key") registered_auth_key = val;
224:           else if (key == "cloud_server") cloud_server = val;
225:           else if (key == "cloud_port") cloud_port = val.toInt();
226:           else if (key == "cloud_user") cloud_user = val;
227:           else if (key == "cloud_pass") cloud_pass = val;
228:         }
229:       }
230:       configFile.close();
231:       Serial.println("[SYSTEM] Configuration loaded successfully.");
232:     }
233:   } else {
234:     Serial.println("[SYSTEM] Configuration file not found. Using defaults.");
235:   }
236: }
237:
238: void saveConfig() {
239:   File configFile = LittleFS.open("/config.txt", "w");
240:   if (configFile) {
241:     configFile.println("wifi_ssid=" + wifi_ssid);
242:     configFile.println("wifi_pass=" + wifi_pass);
243:     configFile.println("node_id=" + node_id);
244:     configFile.println("tailscale_auth_key=" + tailscale_auth_key);
245:     configFile.println("registered_auth_key=" + registered_auth_key);
246:     configFile.println("cloud_server=" + cloud_server);
247:     configFile.println("cloud_port=" + String(cloud_port));
248:     configFile.println("cloud_user=" + cloud_user);
249:     configFile.println("cloud_pass=" + cloud_pass);
250:     configFile.close();
251:     Serial.println("[SYSTEM] Configuration saved successfully.");
252:   } else {
253:     Serial.println("[SYSTEM] Failed to write configuration file.");
254:   }
255: }
```

```
256:
257: // =====
258: // 9. WiFi Portal Configuration Mode
259: // =====
260: void startAPMode() {
```

```
261:   isAPMode = true;
262:   Serial.println("[WIFI] Switching to AP Mode. Stopping active connection attempts...");
263:   WiFi.disconnect(true);
264:   delay(100);
265:   WiFi.mode(WIFI_OFF);
266:   delay(200);
267:   WiFi.mode(WIFI_AP_STA);
268:   delay(200);
269:   String apName = "ESP32-S3-Gateway-" + node_id;
270:   WiFi.softAP(apName.c_str(), "");
271:   delay(100);
272:
273:   Serial.println("[WIFI] Access Point Setup Portal Started!");
274:   Serial.print("[WIFI] SSID: ");
275:   Serial.println(apName);
276:   Serial.print("[WIFI] IP Address: ");
277:   Serial.println(WiFi.softAPIP());
278:
279:   updateOLED("AP Setup Mode");
280: }
281:
282: void setup_wifi() {
283:   loadConfig();
284:
285:   if (wifi_ssid == "") {
286:     Serial.println("[WIFI] No WiFi SSID configured. Entering AP Mode.");
287:     startAPMode();
288:     return;
289:   }
290:
291:   Serial.printf("[WIFI] Connecting to SSID: %s\n", wifi_ssid.c_str());
292:   WiFi.mode(WIFI_STA);
293:   WiFi.begin(wifi_ssid.c_str(), wifi_pass.c_str());
294:
295:   int counter = 0;
296:   updateOLED("Connecting WiFi");
297:   while (WiFi.status() != WL_CONNECTED) {
298:     // Fast blinking Orange/Yellow during Wifi connection
299:     setLedColor(128, 64, 0);
300:     delay(250);
301:     setLedColor(0, 0, 0);
302:     delay(250);
303:     Serial.print(".");
304:     counter++;
305:     if (counter > 30) {
306:       Serial.println("\n[WIFI] Connection failed. Switching to AP Setup Portal.");
307:       startAPMode();
308:       return;
309:     }
310:   }
311:
312:   setLedColor(0, 128, 0); // Solid green for successful connection
313:   Serial.println("\n[WIFI] Connected successfully.");
314:   Serial.print("[WIFI] IP Address: ");
315:   Serial.println(WiFi.localIP());
316:   updateOLED("WiFi Connected");
317: }
318:
319: // =====
320: // 10. Tailscale Setup
```

```
321: // =====
322: void setupTailscale() {
323:   if (tailscale_auth_key == "" || tailscale_auth_key.startsWith("tskey-auth-yourkey")) {
324:     Serial.println("[TAILSCALE] Missing or default Auth Key. Skipping VPN connection.");
325:     return;
```

```
326: }
327:
328: Serial.println("[TAILSCALE] Initializing Tailscale client...");
329:
330: microlink_config_t config;
331: memset(&config, 0, sizeof(config));
332: config.auth_key = tailscale_auth_key.c_str();
333: config.device_name = node_id.c_str();
334: config.enable_derp = true;
335: config.enable_disco = true;
336: config.enable_stun = true;
337: config.max_peers = 16;
338:
339: ml = microlink_init(&config);
340:
341: if (ml != nullptr) {
342:     Serial.println("[TAILSCALE] MicroLink Initialized. Starting VPN Tunnel...");
343:     microlink_start(ml);
344:     isTailscaleStarted = true;
345: } else {
346:     Serial.println("[TAILSCALE] Failed to initialize MicroLink.");
347: }
348: }
349:
350: void printTailscaleStatus() {
351:     if (!isTailscaleStarted || ml == nullptr) return;
352:
353:     static unsigned long lastPrint = 0;
354:     if (millis() - lastPrint > 5000) {
355:         lastPrint = millis();
356:
357:         bool connected = microlink_is_connected(ml);
358:         uint32_t vpn_ip = microlink_get_vpn_ip(ml);
359:
360:         char ip_str[16];
361:         microlink_ip_to_str(vpn_ip, ip_str);
362:
363:         Serial.printf("[TAILSCALE] Status: %s | VPN IP: %s\n",
364:             connected ? "CONNECTED (ONLINE)" : "CONNECTING...", ip_str);
365:     }
366: }
367:
368: // =====
369: // 11. Hardware Debug Indicators & Resets
370: // =====
371: void handleLedIndicator() {
372:     if (isAPMode) {
373:         // Blinking blue in AP configuration mode
374:         static unsigned long lastBlink = 0;
375:         static bool ledState = false;
376:         if (millis() - lastBlink > 500) {
377:             lastBlink = millis();
378:             ledState = !ledState;
379:             if (ledState) setLedColor(0, 0, 128);
380:             else setLedColor(0, 0, 0);
381:         }
382:     } else if (WiFi.status() != WL_CONNECTED) {
383:         // Blinking orange/yellow when disconnected
384:         static unsigned long lastBlink = 0;
385:         static bool ledState = false;
```



```
386:    if (millis() - lastBlink > 500) {  
387:        lastBlink = millis();  
388:        ledState = !ledState;  
389:        if (ledState) setLedColor(128, 64, 0);  
390:        else setLedColor(0, 0, 0);
```

```
391:     }
392:   } else {
393:     // Connected to WiFi. Check Tailscale connection status.
394:     if (isTailscaleStarted && ml != nullptr && microlink_is_connected(ml)) {
395:       setLedColor(0, 128, 128); // Solid Cyan when fully connected online via VPN
396:     } else {
397:       setLedColor(0, 128, 0); // Solid Green when connected to local WiFi but VPN is
inactive/connecting
398:     }
399:   }
400: }
401:
402: void handleResetButton() {
403:   static unsigned long btnPressStart = 0;
404:   static bool btnWasPressed = false;
405:
406:   if (digitalRead(RESET_BTN_PIN) == LOW) {
407:     if (!btnWasPressed) {
408:       btnPressStart = millis();
409:       btnWasPressed = true;
410:       Serial.println("\n[SYSTEM] Reset button pressed. Hold for 3 seconds to clear config...");
411:     } else {
412:       unsigned long pressDuration = millis() - btnPressStart;
413:
414:       if (pressDuration > 3000) {
415:         Serial.println("\n[SYSTEM] Factory Reset Triggered! Erasing configurations...");
416:
417:         // Show immediate visual and screen feedback
418:         updateOLED("Factory Resetting");
419:
420:         // Delete LittleFS configuration file
421:         if (LittleFS.exists("/config.txt")) {
422:           LittleFS.remove("/config.txt");
423:         }
424:
425:         // Wipe local variables
426:         wifi_ssid = "";
427:         wifi_pass = "";
428:         tailscale_auth_key = "";
429:         registered_auth_key = "";
430:
431:         // Reset Tailscale keys in NVS namespaces
432:         microlink_factory_reset();
433:
434:         // Fast blinking red LED indicator
435:         for (int i = 0; i < 20; i++) {
436:           setLedColor(255, 0, 0);
437:           delay(50);
438:           setLedColor(0, 0, 0);
439:           delay(50);
440:         }
441:
442:         Serial.println("[SYSTEM] Configuration erased. Rebooting...");
443:         delay(500);
444:         ESP.restart();
445:       } else {
446:         // Warning fast blink when holding the reset button
447:         static unsigned long lastFastBlink = 0;
448:         if (millis() - lastFastBlink > 100) {
449:           lastFastBlink = millis();
```

```
450:         static bool fs = false;
451:         fs = !fs;
452:         if (fs) setLedColor(255, 128, 0);
453:         else setLedColor(0, 0, 0);
454:     }
455: }
```

```

456:     }
457: } else {
458:     if (btnWasPressed) {
459:         Serial.println("[SYSTEM] Reset button released. Cancelled.");
460:         btnWasPressed = false;
461:         // Restore LED based on connection status
462:         handleLedIndicator();
463:     }
464: }
465: }
466:
467: // =====
468: // 12. HTTP Web API Route Handlers
469: // =====
470: void handleRoot() {
471:     File file = LittleFS.open("/index.html", "r");
472:     if (!file) {
473:         server.send(404, "text/plain", "index.html not found! Please upload LittleFS image.");
474:         return;
475:     }
476:     server.streamFile(file, "text/html");
477:     file.close();
478: }
479:
480: void handleCSS() {
481:     File file = LittleFS.open("/index.css", "r");
482:     if (!file) {
483:         server.send(404, "text/plain", "index.css not found!");
484:         return;
485:     }
486:     server.streamFile(file, "text/css");
487:     file.close();
488: }
489:
490: void handleGetStatus() {
491:     String ipStr = "0.0.0.0";
492:     bool tsConnected = false;
493:
494:     if (ml != nullptr) {
495:         tsConnected = microlink_is_connected(ml);
496:         uint32_t vpn_ip = microlink_get_vpn_ip(ml);
497:         char buf[16];
498:         microlink_ip_to_str(vpn_ip, buf);
499:         ipStr = String(buf);
500:     }
501:
502:     String json = "{";
503:     json += "\"mode\": \"" + String(isAPMode ? "AP" : "STA") + "\", ";
504:
505:     json += "\"config\": {";
506:     json += "\"ssid\": \"" + wifi_ssid + "\", ";
507:     json += "\"node_id\": \"" + node_id + "\", ";
508:     json += "\"tailscale_key_set\": " + String(tailscale_auth_key.length() > 0 ? "true" : "false") + ", ";
509:     json += "\"cloud_server\": \"" + cloud_server + "\", ";
510:     json += "\"cloud_port\": " + String(cloud_port) + ", ";
511:     json += "\"cloud_user\": \"" + cloud_user + "\"";
512:     json += "}, ";
513:
514:     json += "\"tailscale\": {";

```

```
515:  json += "\"connected\":" + String(tsConnected ? "true" : "false") + ",";
516:  json += "\"vpn_ip\":" + ipStr + "\"";
517:  json += "},";
518:
519:  json += "\"nodes\":[\"";
520:  if (!isAPMode) {
```

```

521:     bool firstNode = true;
522:     for (int i = 0; i < MAX_NODES; i++) {
523:         if (nodes[i].active) {
524:             if (!firstNode) json += ",";
525:             firstNode = false;
526:
527:             bool isOnline = (millis() - nodes[i].last_seen < 30000);
528:
529:             json += "{";
530:             json += "\"id\": \"" + nodes[i].id + "\", ";
531:             json += "\"online\": " + String(isOnline ? "true" : "false") + ", ";
532:
533:             json += "\"sensors\":{";
534:             bool firstSensor = true;
535:             for (int j = 0; j < MAX_SENSORS_PER_NODE; j++) {
536:                 if (nodes[i].sensors[j].active) {
537:                     if (!firstSensor) json += ",";
538:                     firstSensor = false;
539:                     json += "\" + nodes[i].sensors[j].key + \": " + String(nodes[i].sensors[j].value);
540:                 }
541:             }
542:             json += "}, ";
543:
544:             json += "\"out\":[";
545:             for (int j = 0; j < 5; j++) {
546:                 json += String(nodes[i].out_states[j] ? 1 : 0);
547:                 if (j < 4) json += ", ";
548:             }
549:             json += "]";
550:             json += "}";
551:         }
552:     }
553: }
554: json += "}]";
555:
556: server.send(200, "application/json", json);
557: }
558:
559: void handleSaveConfig() {
560:     if (!server.hasArg("ssid") || !server.hasArg("node_id") || server.arg("ssid") == "") {
561:         server.send(400, "application/json", "{\"status\": \"error\", \"message\": \"Missing required fields\"}");
562:         return;
563:     }
564:
565:     wifi_ssid = server.arg("ssid");
566:     if (server.hasArg("pass")) wifi_pass = server.arg("pass");
567:     node_id = server.arg("node_id");
568:
569:     // Only overwrite Tailscale key if a new one is provided
570:     if (server.hasArg("tailscale_key") && server.arg("tailscale_key") != "") {
571:         tailscale_auth_key = server.arg("tailscale_key");
572:     }
573:
574:     // Save cloud broker configurations
575:     if (server.hasArg("server")) cloud_server = server.arg("server");
576:     if (server.hasArg("port")) cloud_port = server.arg("port").toInt();
577:     if (server.hasArg("user")) cloud_user = server.arg("user");
578:     if (server.hasArg("cpass")) cloud_pass = server.arg("cpass");
579:

```

```
580:   saveConfig();
581:   server.send(200, "application/json", "{\"status\":\"success\",\"message\":\"Rebooting gateway...\"}");
582:   delay(500);
583:   ESP.restart();
584: }
585:
```

```

586: void handleScanWiFi() {
587:   Serial.println("[WIFI] Scanning for networks...");
588:   int n = WiFi.scanNetworks();
589:   Serial.printf("[WIFI] Scan complete. Found %d networks.\n", n);
590:
591:   String json = "[";
592:   for (int i = 0; i < n; ++i) {
593:     if (i > 0) json += ",";
594:     json += "\"" + WiFi.SSID(i) + "\"";
595:   }
596:   json += "]";
597:
598:   WiFi.scanDelete();
599:   server.send(200, "application/json", json);
600: }
601:
602: void handleControl() {
603:   if (!isAPMode && server.hasArg("node") && server.hasArg("out") && server.hasArg("state")) {
604:     String nodeId = server.arg("node");
605:     int outNum = server.arg("out").toInt();
606:     String stateVal = server.arg("state");
607:
608:     if (outNum >= 1 && outNum <= 5) {
609:       int nodeIdX = -1;
610:       for (int i = 0; i < MAX_NODES; i++) {
611:         if (nodes[i].active && nodes[i].id == nodeId) {
612:           nodeIdX = i;
613:           break;
614:         }
615:       }
616:
617:       if (nodeIdX != -1 && local_broker && cloud_client) {
618:         String cmdTopic = "factory/" + nodeId + "/cmd/out" + String(outNum);
619:         local_broker->publish(cmdTopic.c_str(), stateVal.c_str());
620:
621:         nodes[nodeIdX].out_states[outNum - 1] = (stateVal == "1");
622:
623:         String stateTopic = "factory/" + nodeId + "/state/out" + String(outNum);
624:         cloud_client->publish(stateTopic.c_str(), stateVal.c_str());
625:
626:         server.send(200, "application/json", "{\"status\":\"success\"}");
627:         return;
628:       }
629:     }
630:   }
631:   server.send(400, "application/json", "{\"status\":\"error\"}");
632: }
633:
634: // =====
635: // 13. System Initialization (Setup)
636: // =====
637: void setup() {
638:   Serial.begin(115200);
639:
640:   // Wait for Serial Monitor to open and show countdown
641:   for (int i = 5; i > 0; i--) {
642:     Serial.printf("[SYSTEM] Starting in %d seconds...\n", i);
643:     delay(1000);
644:   }
645:

```



```
646:  // Configure reset button
647:  pinMode(RESET_BTN_PIN, INPUT_PULLUP);
648:
649:  // Initialize RGB LED (off initially)
650:  setLedColor(0, 0, 0);
```

```
651:
652: // Initialize I2C and SSD1306 Display
653: Wire.begin(OLED_SDA_PIN, OLED_SCL_PIN);
654:
655: // Scan I2C bus for debugging
656: Serial.println("\n[I2C] Scanning I2C bus...");
657: byte error, address;
658: int nDevices = 0;
659: for (address = 1; address < 127; address++) {
660:     Wire.beginTransmission(address);
661:     error = Wire.endTransmission();
662:     if (error == 0) {
663:         Serial.printf("[I2C] Device found at address 0x%02X\n", address);
664:         nDevices++;
665:     } else if (error == 4) {
666:         Serial.printf("[I2C] Unknown error at address 0x%02X\n", address);
667:     }
668: }
669: if (nDevices == 0) {
670:     Serial.println("[I2C] No I2C devices found!\n");
671: } else {
672:     Serial.printf("[I2C] Scan complete. Found %d device(s).\n\n", nDevices);
673: }
674:
675: if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
676:     Serial.println(F("[SYSTEM] SSD1306 OLED allocation failed"));
677: } else {
678:     display.clearDisplay();
679:     display.display();
680:     Serial.println(F("[SYSTEM] SSD1306 OLED initialized.));
681: }
682:
683: // Initialize LittleFS
684: if (!LittleFS.begin(true)) {
685:     Serial.println("[SYSTEM] Error mounting LittleFS");
686:     updateOLED("LittleFS Err");
687: } else {
688:     Serial.println("[SYSTEM] LittleFS mounted successfully.");
689: }
690:
691: // Connect to WiFi and load parameters
692: setup_wifi();
693:
694: if (!isAPMode) {
695:     // Check if Tailscale Auth Key has changed
696:     if (tailscale_auth_key != registered_auth_key) {
697:         Serial.println("[TAILSCALE] Auth Key change detected. Resetting NVS keys...");
698:         updateOLED("Resetting VPN");
699:         microlink_factory_reset();
700:         registered_auth_key = tailscale_auth_key;
701:         saveConfig();
702:     }
703:
704:     if (MDNS.begin(node_id.c_str())) {
705:         Serial.printf("[mDNS] Hostname Registered: %s.local\n", node_id.c_str());
706:         MDNS.addService("mqtt", "tcp", 1883);
707:         MDNS.addService("http", "tcp", 80);
708:     }
709:
710:     // Setup Tailscale VPN Client
```

```
711:     setupTailscale();
712:
713:     // Initialize PicoMQTT Local Broker and Cloud Client Bridge
714:     local_broker = new PicoMQTT::Server(1883);
715:     cloud_client = new PicoMQTT::Client(cloud_server.c_str(), cloud_port, nullptr,
cloud_user.c_str(), cloud_pass.c_str());
```

```

716:
717:     local_broker->subscribe("factory/#", [](const char* topic, const char* payload) {
718:         String t = topic;
719:         String p = payload;
720:
721:         if (t.startsWith("factory/")) {
722:             int secondSlash = t.indexOf('/', 8);
723:             if (secondSlash != -1) {
724:                 String nodeId = t.substring(8, secondSlash);
725:                 String subTopic = t.substring(secondSlash + 1);
726:
727:                 int idx = getOrRegisterNode(nodeId);
728:                 if (idx != -1) {
729:                     // Check dynamic sensors: factory/<node_id>/sensor/<sensor_key>
730:                     if (subTopic.startsWith("sensor/")) {
731:                         String sensorKey = subTopic.substring(7);
732:                         float val = p.toFloat();
733:                         updateNodeSensor(idx, sensorKey, val);
734:                     }
735:                     // Check output states
736:                     else if (subTopic.startsWith("state/out")) {
737:                         int outNum = subTopic.substring(9).toInt();
738:                         if (outNum >= 1 && outNum <= 5) {
739:                             nodes[idx].out_states[outNum - 1] = (p == "1" || p == "ON" || p == "on");
740:                         }
741:                     }
742:                 }
743:             }
744:         }
745:
746:         // Forward to Cloud Client
747:         if (cloud_client && (t.indexOf("/sensor/") != -1 || t.indexOf("/state/") != -1)) {
748:             cloud_client->publish(topic, payload);
749:         }
750:     });
751:
752:     cloud_client->subscribe("factory/+ /cmd/#", [](const char* topic, const char* payload) {
753:         // Forward to Local Broker
754:         if (local_broker) {
755:             local_broker->publish(topic, payload);
756:         }
757:
758:         String t = topic;
759:         if (t.startsWith("factory/")) {
760:             int secondSlash = t.indexOf('/', 8);
761:             if (secondSlash != -1) {
762:                 String nodeId = t.substring(8, secondSlash);
763:                 String subTopic = t.substring(secondSlash + 1);
764:
765:                 int idx = getOrRegisterNode(nodeId);
766:                 if (idx != -1 && subTopic.startsWith("cmd/out")) {
767:                     int outNum = subTopic.substring(7).toInt();
768:                     if (outNum >= 1 && outNum <= 5) {
769:                         nodes[idx].out_states[outNum - 1] = (strcmp(payload, "1") == 0);
770:                         if (cloud_client) {
771:                             String stateTopic = "factory/" + nodeId + "/state/out" + String(outNum);
772:                             cloud_client->publish(stateTopic.c_str(), payload);
773:                         }
774:                     }
775:                 }

```

```
776:         }  
777:     }  
778: });  
779:  
780:     local_broker->begin();
```

```
781:     cloud_client->begin();
782:     updateOLED("Ready");
783: }
784:
785: // Setup Web Server Endpoints
786: server.on("/", HTTP_GET, handleRoot);
787: server.on("/index.css", HTTP_GET, handleCSS);
788: server.on("/api/status", HTTP_GET, handleGetStatus);
789: server.on("/api/save_config", HTTP_POST, handleSaveConfig);
790: server.on("/api/save_config", HTTP_GET, handleSaveConfig);
791: server.on("/api/scan", HTTP_GET, handleScanWiFi);
792: server.on("/api/control", HTTP_POST, handleControl);
793: server.on("/api/control", HTTP_GET, handleControl);
794:
795: server.begin();
796: Serial.println("[SYSTEM] HTTP Web Server started.");
797: }
798:
799: void checkConnectionRecovery() {
800:     if (isAPMode) return;
801:
802:     static unsigned long lastCheck = 0;
803:     static unsigned long disconnectTime = 0;
804:     static bool wasConnected = true;
805:     unsigned long now = millis();
806:
807:     if (now - lastCheck < 5000) return; // 5
808:     lastCheck = now;
809:
810:     if (WiFi.status() == WL_CONNECTED) {
811:         if (!wasConnected) {
812:             Serial.println("[SYSTEM] WiFi Reconnected successfully!");
813:             updateOLED("WiFi Recovered");
814:
815:             // Rebind VPN Sockets Tailscale
816:             if (isTailscaleStarted && ml != nullptr) {
817:                 Serial.println("[TAILSCALE] Rebinding VPN sockets to new connection...");
818:                 esp_err_t err = microlink_rebind(ml);
819:                 if (err == ESP_OK) {
820:                     Serial.println("[TAILSCALE] VPN Rebind successful.");
821:                 } else {
822:                     Serial.printf("[TAILSCALE] VPN Rebind failed: %d. Restarting Tailscale...\n", err);
823:                     microlink_stop(ml);
824:                     setupTailscale();
825:                 }
826:             }
827:             wasConnected = true;
828:             disconnectTime = 0;
829:         }
830:     } else {
831:         // WiFi
832:         if (wasConnected) {
833:             Serial.println("[SYSTEM] WiFi Disconnected!");
834:             disconnectTime = now;
835:             wasConnected = false;
836:             updateOLED("WiFi Lost");
837:         }
838:
839:         unsigned long offlineDuration = now - disconnectTime;
840:         Serial.printf("[SYSTEM] WiFi is offline for %lu seconds. Attempting reconnect...\n",
```

```
offlineDuration / 1000);  
841:  
842:    // 15  
843:    if (offlineDuration % 15000 < 5000) {  
844:        WiFi.begin(wifi_ssid.c_str(), wifi_pass.c_str());  
845:    }
```

```
846:
847: // 60 Hard Reset WiFi Stack Driver
848: if (offlineDuration > 60000) {
849:     Serial.println("[SYSTEM] WiFi offline for >60s. Hard resetting WiFi stack...");
850:     WiFi.disconnect(true);
851:     delay(200);
852:     WiFi.mode(WIFI_OFF);
853:     delay(200);
854:     WiFi.mode(WIFI_STA);
855:     WiFi.begin(wifi_ssid.c_str(), wifi_pass.c_str());
856:     disconnectTime = now; //
857: }
858: }
859: }
860:
861: // =====
862: // 14. Main Execution Loop
863: // =====
864: void loop() {
865:     handleLedIndicator();
866:     handleResetButton();
867:
868:     if (!isAPMode) {
869:         checkConnectionRecovery();
870:         if (local_broker) local_broker->loop();
871:         if (cloud_client) cloud_client->loop();
872:
873:         printTailscaleStatus();
874:
875:         static unsigned long lastOledUpdate = 0;
876:         if (millis() - lastOledUpdate > 5000) {
877:             lastOledUpdate = millis();
878:             updateOLED("Monitoring");
879:         }
880:     } else {
881:         static unsigned long lastOledUpdate = 0;
882:         if (millis() - lastOledUpdate > 3000) {
883:             lastOledUpdate = millis();
884:             updateOLED("AP Config Open");
885:         }
886:     }
887:
888:     server.handleClient();
889: }
```


6. การคอมไพล์และการอัปโหลดโปรแกรม (Flashing & Uploading)

ในการเชื่อมต่อและทำการเขียนข้อมูลลงไปยังชิป ESP32-S3 ให้ทำตามขั้นตอนดังต่อไปนี้:

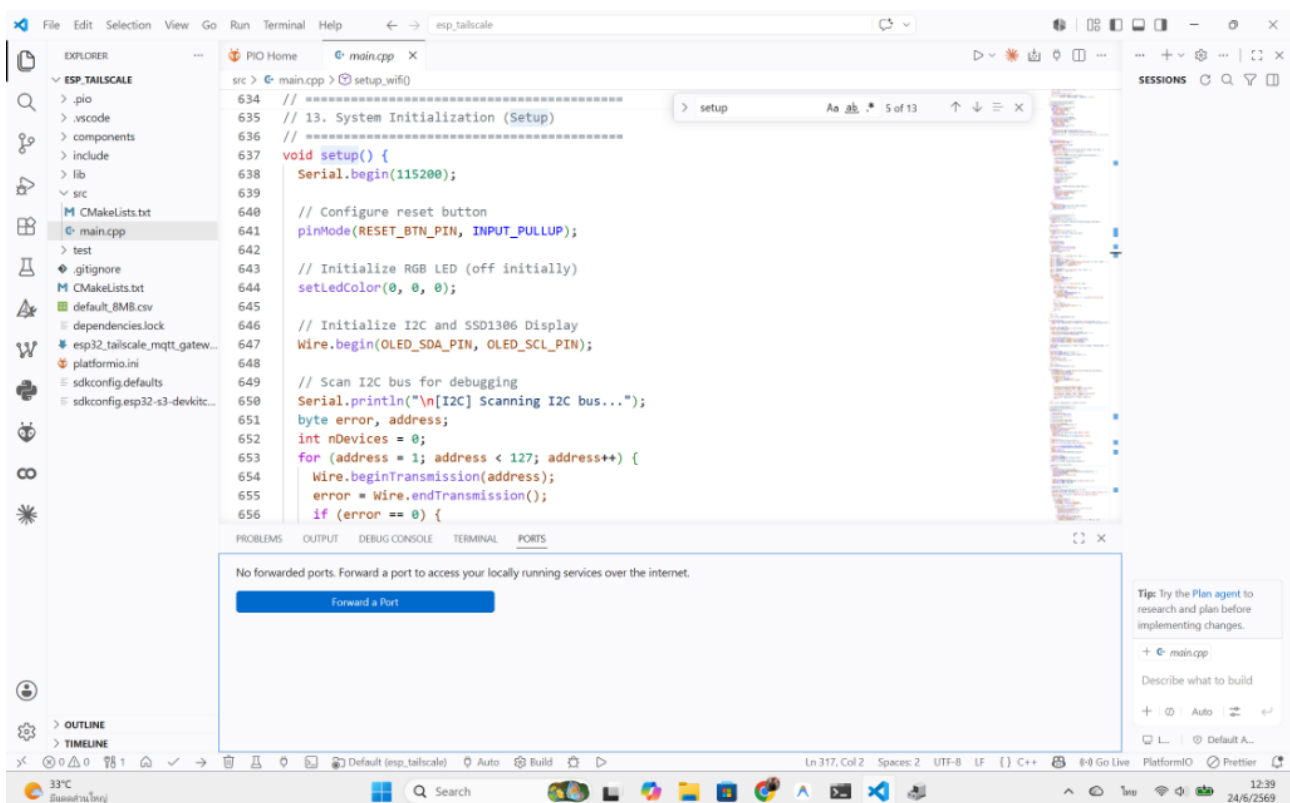
1. ทำการต่อพอร์ต USB จากคอมพิวเตอร์เข้ากับช่อง USB (หรือพอร์ต UART) ของบอร์ด ESP32-S3 DevKit
2. ตรวจสอบหมายเลข COM Port ของบอร์ดในคอมพิวเตอร์ผ่าน Device Manager (ตัวอย่างเช่น COM4)
3. รันคำสั่งคอมไพล์เฟิร์มแวร์เพื่อสร้างไฟล์ระบบไบนารี:
> pio run
4. รันคำสั่งเขียนโปรแกรมลงชิป โดยเจาะจงพอร์ตการเชื่อมต่อที่กำลังใช้งาน (ในที่นี้คือพอร์ต COM4):
> pio run -t upload --upload-port COM4
5. หากต้องการใช้ฟังก์ชันการทำงานของ Web Server จากหน้าเว็บ LittleFS จะต้องทำการสร้างและอัปโหลดไฟล์ HTML/CSS ไปด้วย โดยเตรียมข้อมูลในโฟลเดอร์ data/ และพิมพ์คำสั่งดังนี้:
> pio run -t uploadfs --upload-port COM4

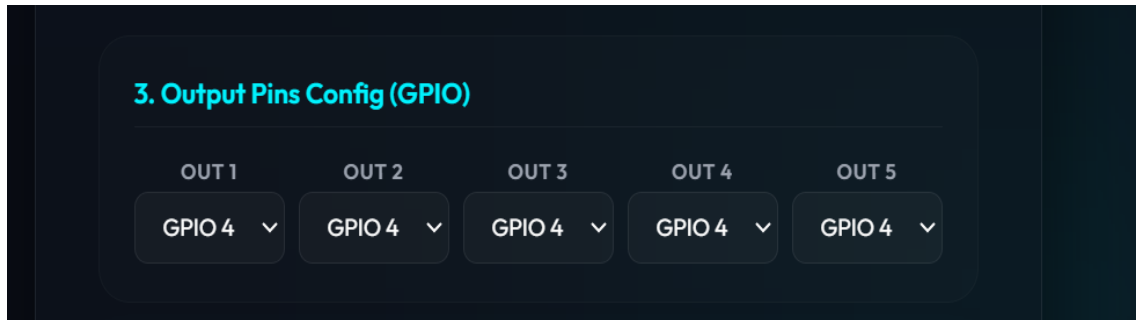
7. ภาพประกอบและผลการทำงานจริง (Visual Verification)

รูปภาพด้านล่างนี้คือขั้นตอนการพัฒนาและผลการตรวจสอบความถูกต้องของการเชื่อมต่อพอร์ตต่าง ๆ และหน้าสัมผัสของระบบจัดการขาดังกล่าว:

๑

รูปภาพที่ 1: หน้าจอ VS Code แสดงโครงสร้างโปรเจกต์และฟังก์ชัน setup()



รูปภาพที่ 2: ระบบจัดการและปรับแต่งพิน GPIO เอาต์พุต (Relay Interface) ในหน้าเว็บ UI

กระบวนการและโค้ดทั้งหมดที่แสดงในคู่มือนี้ ได้รับการออกแบบ ปรับปรุง และคอมไพล์ผ่านบนสภาพแวดล้อมใช้งานจริง และตัวอุปกรณ์สามารถตรวจจับสัญญาณเน็ตเวิร์กที่หลุดกลางทาง พร้อมสับเปลี่ยนพอร์ตและพินตัวเชื่อมต่อกับ VPN ของ Tailscale โดยอัตโนมัติ ช่วยลดปัญหาอุปกรณ์ค้างและต้องเข้าถึงเพื่อรีเซ็ตทางกายภาพได้ 100%